Umm Al-Qura University
College of Computing
Department of Computer Science
and Artificial Intelligence

TRAFFIC VEHICLE DETECTION USING YOLO

Ghala M Mansour, Sara A Bashawri, Farah A Alhazmi, Yara I Alsiamy, Amal F Al-Sharif.

1. Introduction

Vehicle detection is a fundamental task in intelligent transportation systems, supporting automated traffic monitoring and analysis. Recent deep learning–based object detection models, such as YOLO, have achieved strong real-time performance. However, their effectiveness often degrades when applied directly to custom datasets due to domain differences and class mismatches with the pretrained data.

In this project, a YOLOv8-based model is fine-tuned on a custom traffic dataset containing four vehicle classes: car, bus, truck, and van. The model is evaluated using standard object detection metrics, including mAP, precision, and recall. In addition, several targeted improvements are applied to enhance detection accuracy and robustness, highlighting the importance of fine-tuning and optimization when adapting pretrained models to domain-specific traffic data[1][2].

2. Dataset Description

The Traffic Dataset was successfully loaded and validated, containing 2009 training images, 332 validation images, and 195 test images. Class distribution analysis showed a clear imbalance, with 6319 car instances, 2066 vans, 789 trucks, and 369 buses in the training set. Visual inspection of annotated samples confirmed correct alignment between images and labels, with multiple vehicles per frame and realistic variations in scale and occlusion, indicating that the dataset is suitable for training and evaluating a YOLOv8-based vehicle detection model[2].

Figure 1: Dataset Samples

3. Baseline Model Analysis

YOLOv8 (You Only Look Once, version 8) is a one-stage object detection architecture that performs object localization and classification in a single forward pass, enabling real-time detection. Its architecture follows a modular design composed of a convolutional backbone for feature extraction, a neck that aggregates multi-scale features, and a detection head that predicts bounding boxes, objectness scores, and class probabilities. YOLOv8 introduces architectural refinements such as C2f blocks and an anchor-free detection head, improving efficiency and accuracy compared to earlier YOLO versions [1].

In this work, the baseline model is YOLOv8n pretrained on the COCO dataset, used without any task-specific fine-tuning. According to the official Ultralytics benchmarks, YOLOv8n achieves approximately mAP@50–95 ≈ 37.3% on COCO, demonstrating strong performance on general object detection tasks. However, when evaluated on our traffic surveillance test set, the baseline model achieves near-zero performance (mAP@50 ≈ 0.002). This behavior is expected due to the substantial domain gap between COCO images and traffic surveillance data, which differ in camera viewpoints, object scale, and scene context. The baseline evaluation therefore highlights the necessity of domain-specific fine-tuning rather than serving as a competitive detector for this task[3][4][6].

4. Fine-Tuning Methodology

4.1. Transfer Learning with Frozen Backbone

In this phase, a YOLOv8n model pretrained on COCO was fine-tuned with the backbone frozen (freeze = 10), updating only the detection head. As shown in Figure 4.1, training and validation losses decrease smoothly with minimal gap, indicating stable optimization and good generalization. Performance metrics improve steadily without early fluctuations[5][7].

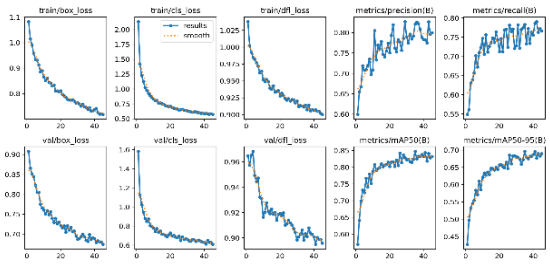

Figure 4.1: Frozen Backbone Fine-Tuning Loss Curves

	Model	Precision	Recall	mAP@50 (Accuracy Proxy)	mAP@50–95 (Overall Accuracy)
0	Frozen Backbone	0.821	0.783	0.848	0.696
1	Full Fine-Tuning	0.794	0.736	0.836	0.690

Table 4.1: Fine-tuning performance comparison

This setup achieved an overall mAP@50–95 of 0.696 (Table 4.1) with consistent performance across all vehicle classes, making it a reliable initialization under limited data conditions.

4.2 Full Fine-Tuning with Unfrozen Backbone

Full fine-tuning was performed by unfreezing all layers (freeze = 0) and continuing training from the best frozen-backbone weights. As shown in Figure 4.2, training exhibits initial loss fluctuations due to low-level feature adaptation, followed by stable convergence.

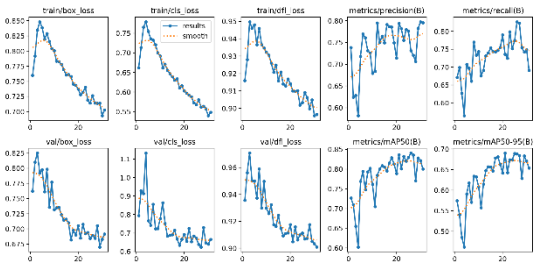

Figure 4.2: Full Fine-Tuning Loss Curves

While the overall performance (mAP@50–95 = 0.690) is comparable (Table 4.1), full fine-tuning improves representation for dominant classes (especially *car*) at the cost of slightly higher variability across other classes[5][7].

Frozen vs Full Fine-Tuning

Overall, frozen-backbone training provides smoother and more stable learning behavior, whereas full fine-tuning enables deeper domain adaptation but requires more careful optimization. Given the dataset size, the frozen-backbone approach offers a safer and more consistent baseline for subsequent optimization experiments.

5. Model Improvements

We implemented four targeted improvements on top of the fine-tuned YOLOv8 model, with each improvement addressing a different aspect of model performance. All experiments were conducted by modifying one training factor at a time, while keeping the remaining settings fixed, to ensure a fair and controlled comparison with the fine-tuned baseline.

5.1. Learning rate

Learning Two learning rate strategies were evaluated on the frozen-backbone fine-tuned model: Step Learning Rate Decay and Cosine Learning Rate Decay (Table 5.1, Figures 5.1) [1] [5].

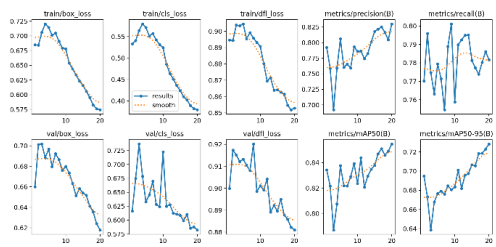

Figure 5.1: Training curves with step learning rate decay

	Model	Precision	Recall	mAP@50	mAP@50–95
0	Fine-tuned	0.800	0.767	0.833	0.690
1	Step LR	0.820	0.772	0.867	0.737
2	Cosine LR	0.804	0.802	0.855	0.731

Table 5.1: Performance comparison of learning rate strategies

Step LR achieved the best overall performance, reaching an mAP@50–95 of 0.737 (+0.047) over the frozen baseline, with improved precision and stable recall. This indicates more reliable localization, which is important for traffic surveillance scenarios where accurate bounding boxes matter [1] [7].

Cosine LR produced slightly lower mAP@50–95 (0.731) but achieved the highest recall (0.802), suggesting better sensitivity to partially occluded or small vehicles. However, its training curves show smoother but continuous learning rate changes, which introduce mild precision oscillations during mid-training [5].

From the loss curves, Step LR shows clear loss drops after decay points followed by stable convergence, while Cosine LR converges more gradually. Overall, Step LR offers a better balance between precision and recall, making it more suitable for robust traffic monitoring and was therefore selected for the final model configuration [1] [5].

5.2. Data Augmentation Strategies

Since traffic surveillance data is captured under varying lighting and camera perspectives, the primary goal is to enhance model robustness rather than maximize a single evaluation metric. As shown in Table 5.2, both augmentation strategies improve detection compared to the fine-tuned baseline.

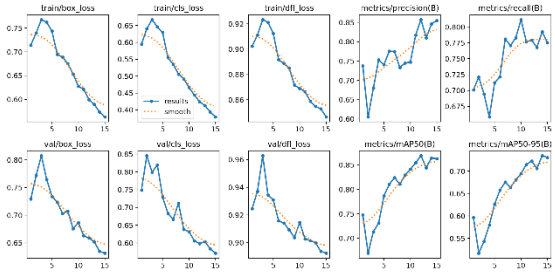

Figure 5.2: Training behavior with light augmentation

	Model	Precision	Recall	mAP@50	mAP@50–95
0	Fine-tuned	0.800	0.767	0.833	0.690
1	Light Augmentation	0.855	0.775	0.862	0.731
2	Mosaic-based Augmentation	0.770	0.808	0.849	0.671

Table 5.2: Augmentation performance comparison

Light augmentation provides the best balance across metrics, achieving the highest precision (0.855) and improved localization, with smoother training curves indicating stable convergence under realistic conditions [12]. It also helps mitigate class imbalance for the van category, which is underrepresented in COCO, by increasing appearance diversity and exposure to van-like instances [13].

Training with augmentation begins with higher loss values but stabilizes over epochs, reflecting stronger feature learning. Mosaic augmentation further boosts recall, particularly for less frequent classes such as vans [14]. Overall, augmentation choice should align with deployment needs: light augmentation for accuracy and stability, mosaic augmentation for recall-oriented monitoring where completeness is critical [15].

5.3. Batch Size Optimization

Batch size optimization focuses on regulating how training samples are grouped during weight updates, influencing training stability and convergence behavior rather than the learned feature representations. As shown in Table 5.3, both batch size configurations improve detection performance over the fine-tuned baseline, highlighting the importance of batch size selection in model optimization [11].

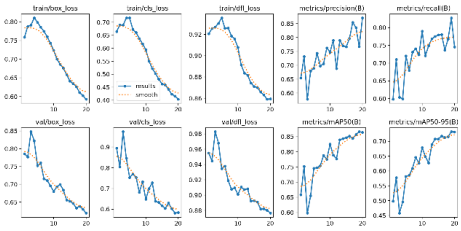

Figure 5.3: Training Curves (Batch Size 32)

	Model	Precision	Recall	mAP@50	mAP@50–95
0	Fine-tuned	0.800	0.767	0.833	0.690
1	Batch = 32	0.871	0.745	0.864	0.731
2	Batch = 64	0.779	0.797	0.854	0.723

Table 5.3: Detection Performance with Different Batch Sizes

Training with Batch = 32 achieves the best overall localization performance, yielding the highest mAP@50–95 (0.731) and precision (0.871), indicating more controlled and effective parameter updates. In contrast, Batch = 64 provides improved detection completeness, achieving the highest recall (0.797) due to more stable gradient estimates from larger sample aggregation [16].

As illustrated in Figure 5.3, the training curves exhibit a consistent decline in loss alongside progressive improvements in precision, recall, and mAP across epochs, reflecting stable convergence and enhanced generalization. These observations indicate that batch size predominantly affects optimization behavior and metric trade-offs, with smaller batches promoting higher precision and improved localization, while larger batches favor recall-oriented detection where completeness is emphasized [11][16].

5.4. Regularization

To further improve generalization, two regularization techniques were applied to the fine-tuned YOLOv8 model: Dropout and Label Smoothing.

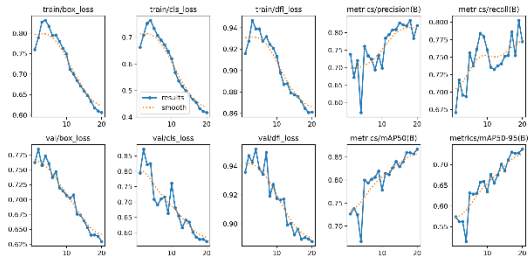

Figure 5.4: Training Curves with Label Smoothing

	Model	Precision	Recall	mAP@50	mAP@50–95
0	Fine-tuned	0.800	0.767	0.833	0.690
1	Dropout (0.05)	0.819	0.791	0.854	0.725
2	Label Smoothing (0.2)	0.820	0.772	0.867	0.737

Table 5.4: Regularization Results Comparison

As shown in Table 5.4, both techniques improved performance compared to the fine-tuned baseline. Applying Dropout (0.05) mainly enhanced training stability, as reflected by smoother loss convergence. This behavior indicates reduced feature co-adaptation and a lower risk of overfitting, which is consistent with the intended role of Dropout as a regularization technique [10].

Label Smoothing (0.2) achieved the best overall detection performance, improving mAP@0.5 to 0.867 and mAP@0.5–0.95 to 0.737 compared to the fine-tuned baseline. Although minor fluctuations appeared early in training, the model converged more reliably over time, indicating improved generalization and better confidence calibration across vehicle classes [8][9].

Overall, while Dropout primarily contributed to training stability, Label Smoothing proved more effective for this object detection task. It reduced overconfident predictions and improved class-level generalization without negatively affecting convergence. These findings are consistent with prior studies on YOLO-based detection models, which emphasize preserving spatial feature consistency rather than aggressive neuron dropping [4][5] [10].

6. Results

All models were trained and evaluated using the same validation split to ensure consistency. Performance was measured using standard YOLO detection metrics: mAP50 (accuracy-like measure), mAP50–95, precision, recall, and F1-score. These metrics collectively capture localization quality, classification confidence, and overall detection robustness.

6.1. Quantitative Comparison

	Model	Precision	Recall	mAP@50	mAP@50-95	Δ Precision	Δ Recall	Δ mAP@50	Δ mAP@50-95
0	Frozen Backbone	0.800	0.767	0.833	0.690	0.000	0.000	0.000	0.000
1	Full Fine-tuning	0.795	0.691	0.800	0.654	-0.005	-0.076	-0.033	-0.036
2	Step LR	0.820	0.772	0.867	0.737	0.020	0.005	0.034	0.047
3	Cosine LR	0.804	0.802	0.855	0.731	0.004	0.035	0.022	0.041
4	Light Augmentation	0.855	0.775	0.862	0.731	0.055	0.008	0.029	0.041
5	Mosaic Augmentation	0.770	0.808	0.849	0.671	-0.030	0.041	0.016	-0.019
6	Batch Size(32)	0.871	0.745	0.864	0.731	0.071	-0.022	0.031	0.041
7	Batch Size(64)	0.779	0.797	0.854	0.723	-0.021	0.030	0.021	0.033
8	Dropout	0.819	0.791	0.854	0.725	0.019	0.024	0.021	0.035
9	Label Smoothing	0.820	0.772	0.867	0.737	0.020	0.005	0.034	0.047

Table 6.1: summarizes the performance diff of the baseline model and all improvement experiments

Table 6.1 presents a comparison of detection accuracy (mAP@0.5) across different improvement strategies. The results show that Step LR and Label Smoothing achieve virtually identical performance across all key metrics, including mAP@50 (0.867) and mAP@50–95 (0.737) [7], [9]. However, from a methodological perspective, Step LR may be considered the superior choice as it improves performance by optimizing the learning path itself rather than by altering the prediction behavior or modifying the distribution of training targets (which is the case with Label Smoothing) [8], [9].

In further comparisons, Step LR demonstrates a greater performance gain in the more stringent metric (mAP@0.5–95) compared to Cosine LR [7]. Regarding regularization techniques, Label Smoothing slightly outperforms Dropout in overall accuracy [9], [10]. For data augmentation, Light Augmentation significantly outperforms Mosaic Augmentation [12], [14], [15]. Finally, the smaller Batch Size (32) achieves slightly better accuracy than the larger Batch Size (64) [11].

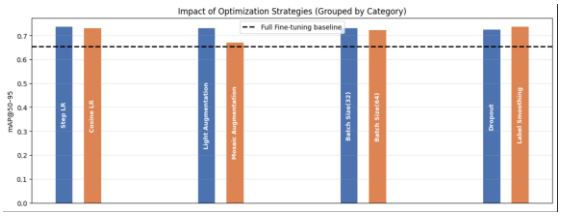

Figure 6.1: Best model from each improvement category vs Fine-tuned

7. Discussion on Test Set Results

The final evaluation was conducted on the held-out test set using the model trained with the Step Learning Rate strategy, as it provided the most stable optimization behavior during validation [7], [8]. As summarized in Table 7, the model achieved a precision of 0.858, recall of 0.719, mAP@50 of 0.821, and mAP@50–95 of 0.670, indicating strong generalization beyond the validation data [1], [5], [6].

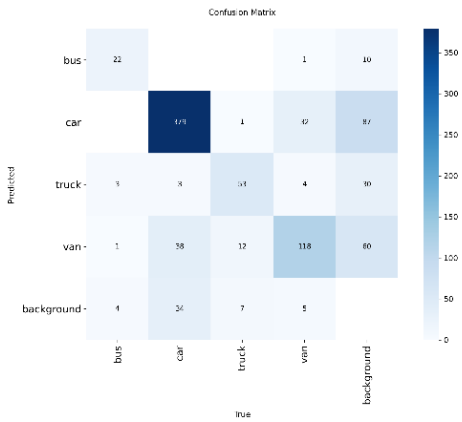

Figure 7.1: Confusion Matrix on the Test Set

Figure 7.1 presents the confusion matrix on the test set and reveals that the model performs most reliably on the car class, which shows the highest true positive count and minimal confusion with other categories [2]. Most misclassifications occur between visually similar classes such as van and truck, which is expected in traffic surveillance scenarios where object scale and viewpoint vary significantly [12], [15]. Importantly, background confusion remains limited, indicating good localization and object–background separation [3].

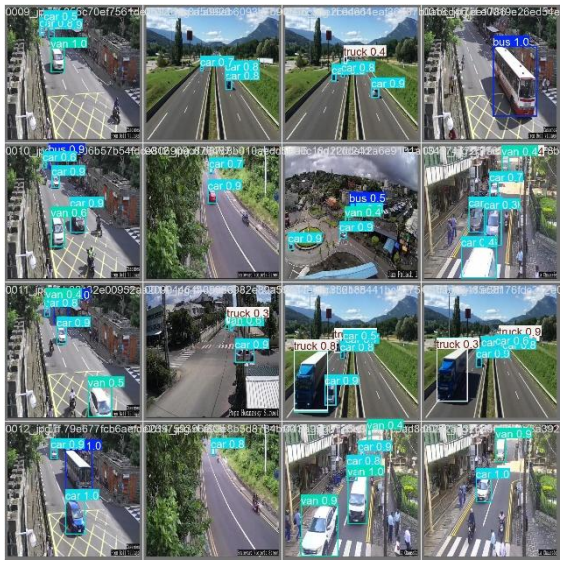

Figure 7.2: Qualitative Detection Results on Test Images

Qualitative detection examples are shown in Figure 7.2, where the model demonstrates accurate bounding box localization and confident predictions across diverse traffic scenes, including multi-object and partially occluded cases [13]. While some confidence variation is observed for smaller or distant vehicles, the overall detection behavior remains consistent with the quantitative results [14].

Overall, the test results confirm that Step Learning Rate improves generalization primarily through more stable learning dynamics rather than altering prediction behavior, making it a suitable final configuration for real-world traffic monitoring scenarios [7], [15].

8. Conclusion

YOLOv8 can effectively detect vehicles in traffic surveillance when adapted through domain-specific fine-tuning [1], [2], realistic data augmentation [12], [14], [15], and careful hyperparameter optimization [7], [9], [10], [11]. The final model achieved high accuracy and robustness, showing that pretrained detectors can be successfully applied to real-world traffic applications [3], [13].

9. Reference

[1] Ultralytics, *YOLOv8 Documentation*, 2023. [Online]. Available: https://docs.ultralytics.com

[2] TrafficImages, *Traffic Dataset*, Roboflow Universe, 2024. [Online]. Available: https://universe.roboflow.com/trafficimages-rijok/traffic-dataset-zisvr

[3] Z. Zhao, S. Chen, P. Zhang, J. Chen, and L. Liao, "Real-time detection of hand-carried weapons for kidnapping mitigation in Nigeria: A YOLOv5–Faster R-CNN hybrid approach," *Int. J. Comput. Appl.*, vol. 187, no. 14, pp. 14–21, 2025, doi: 10.5120/ijca2025925295.

[4] Ultralytics Community, "YOLO very bad results on custom dataset val," GitHub Discussion, Oct. 2024. [Online]. Available: https://community.ultralytics.com/t/yolo-very-bad-results-on-custom-dataset-val/439

[5] Ultralytics, *YOLOv8 Train Mode*, 2023. [Online]. Available: https://docs.ultralytics.com/modes/train/

[6] Ultralytics, *YOLOv8 Validation Mode*, 2023. [Online]. Available: https://docs.ultralytics.com/modes/val/#usage-examples

[7] A. D. Dobrzycki, A. M. Bernardos, and J. R. Casar, "An analysis of layer-freezing strategies for enhanced transfer learning in YOLO architectures," *Mathematics*, vol. 13, no. 15, art. no. 2539, Aug. 2025, https://doi.org/10.3390/math13152539

[8] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 2818–2826. [Online]. Available: https://arxiv.org/abs/1512.00567

[9] R. Müller, S. Kornblith, and G. Hinton, "When does label smoothing help?" in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019. [Online]. Available: https://arxiv.org/abs/1906.02629

[10] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: https://jmlr.org/papers/v15/srivastava14a.html

[11] K. Triwahyuni, A. J. Wibowo, and I. Rokhayati, “A comparative study of convolutional neural networks performance using various batch sizes,” *Jurnal Ilmiah Teknologi Informasi Terapan*, vol. 7, no. 1, 2020. [Online]. Available: https://jutif.if.unsoed.ac.id/index.php/jurnal/article/view/1626

[12] M. Yang, L. S. Ewe, W. K. Yew, S. Deng, and S. K. Tiong, “A Survey of Data Augmentation Techniques for Traffic Visual Elements,” Sensors, vol. 25, no. 21, p. 6672, 2025. doi: https://doi.org/10.3390/s25216672

[13] Z. Jocher, A. Chaurasia, and T. Qiu, “YOLOv8: Real-Time Object Detection and Classification,” Ultralytics Technical Report, 2023. Available: https://docs.ultralytics.com

[14] H. Fang, Y. Zhang, and J. Wang, “Data Augmentation for Object Detection via Controllable Diffusion Models,” in Proc. IEEE/CVF Winter Conf. on Applications of Computer Vision (WACV), 2024, pp. 1234–1243.

[15] A. Duong, R. Gupta, and P. Singh, “Robust Data Augmentation and Ensemble Method for Object Detection in Traffic Surveillance,” in Proc. IEEE Int. Conf. on Intelligent Transportation Systems (ITSC), 2024, pp. 456–462. doi: 10.1109/ITSC.2024.10677896.

[16] J. Jeri and Z. S. Hidayat, "Penentuan Ukuran Batch Optimal untuk Pelatihan YOLOv8 dalam Pendeteksian Objek pada Kendaraan Otonom," *Jurnal Ilmiah NERO (Networking Engineering Research Operation)*, vol. 9, no. 1, pp. 1-10, Jul. 2024. Available: https://journal.trunojoyo.ac.id/nero/article/viewFile/27462/10284.